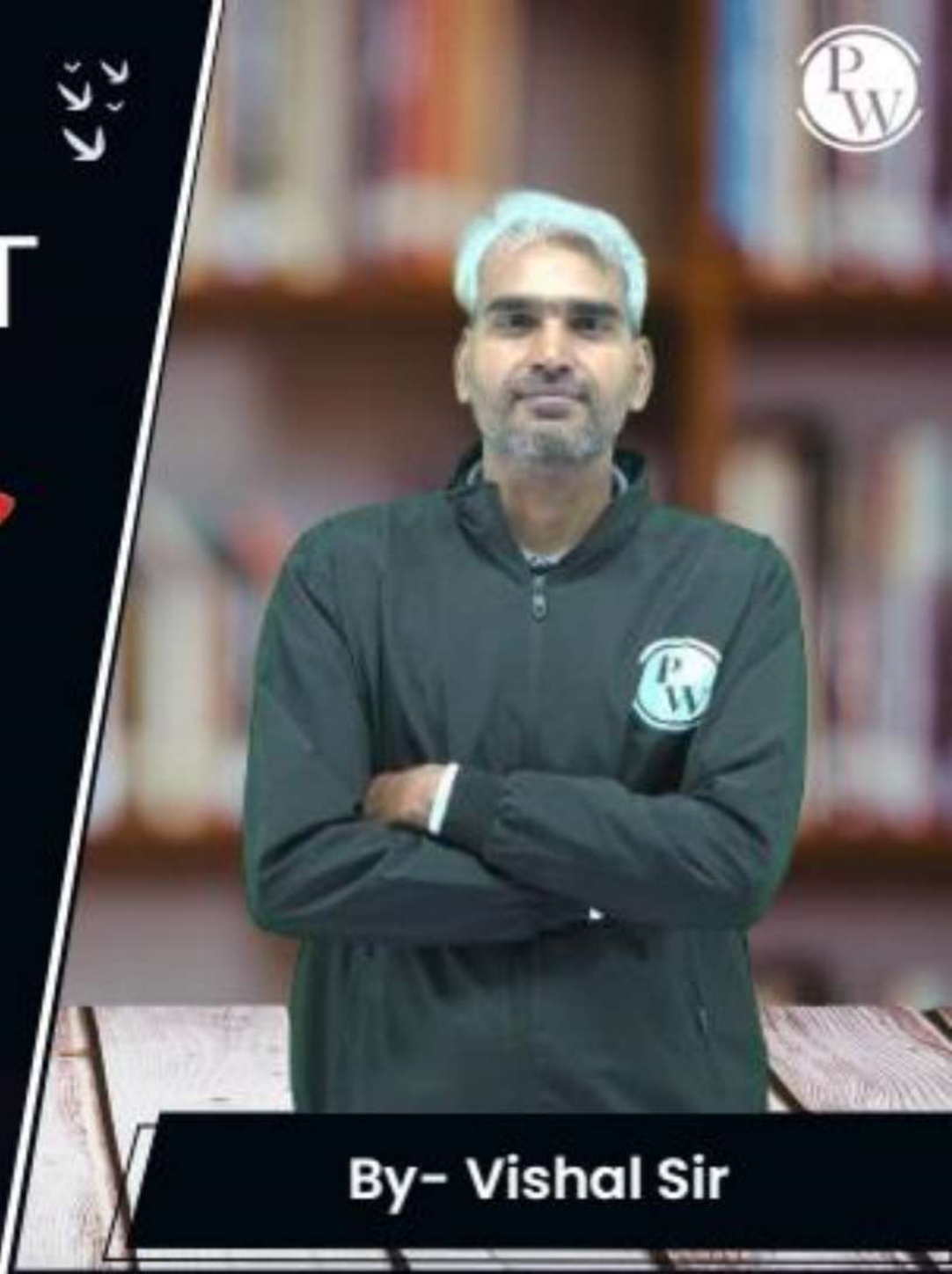
Computer Science & IT



Compiler Design

Introduction and Lexical Analysis

Lecture No. 04



By- Vishal Sir

Recap of Previous Lecture



Topic — Symbol Table & Error Handler

Topic — Translation example

Slide

Topics to be Covered



Topic Translation example (continue..)

Topic Lexical Analyzer

Slide



Topic : Translation example

Consider the following grammar

$S \rightarrow id=E$

$E \rightarrow E+T \mid T$

$T \rightarrow T*F \mid F$

$F \rightarrow id$

And also consider the string $x = a + b * 10$ generated by the grammar

Lexical Analysis :

x $=$ a $+$ b $*$ 10 } 7 tokens

Slide

Type checking is not performed by Syntax Analyzer

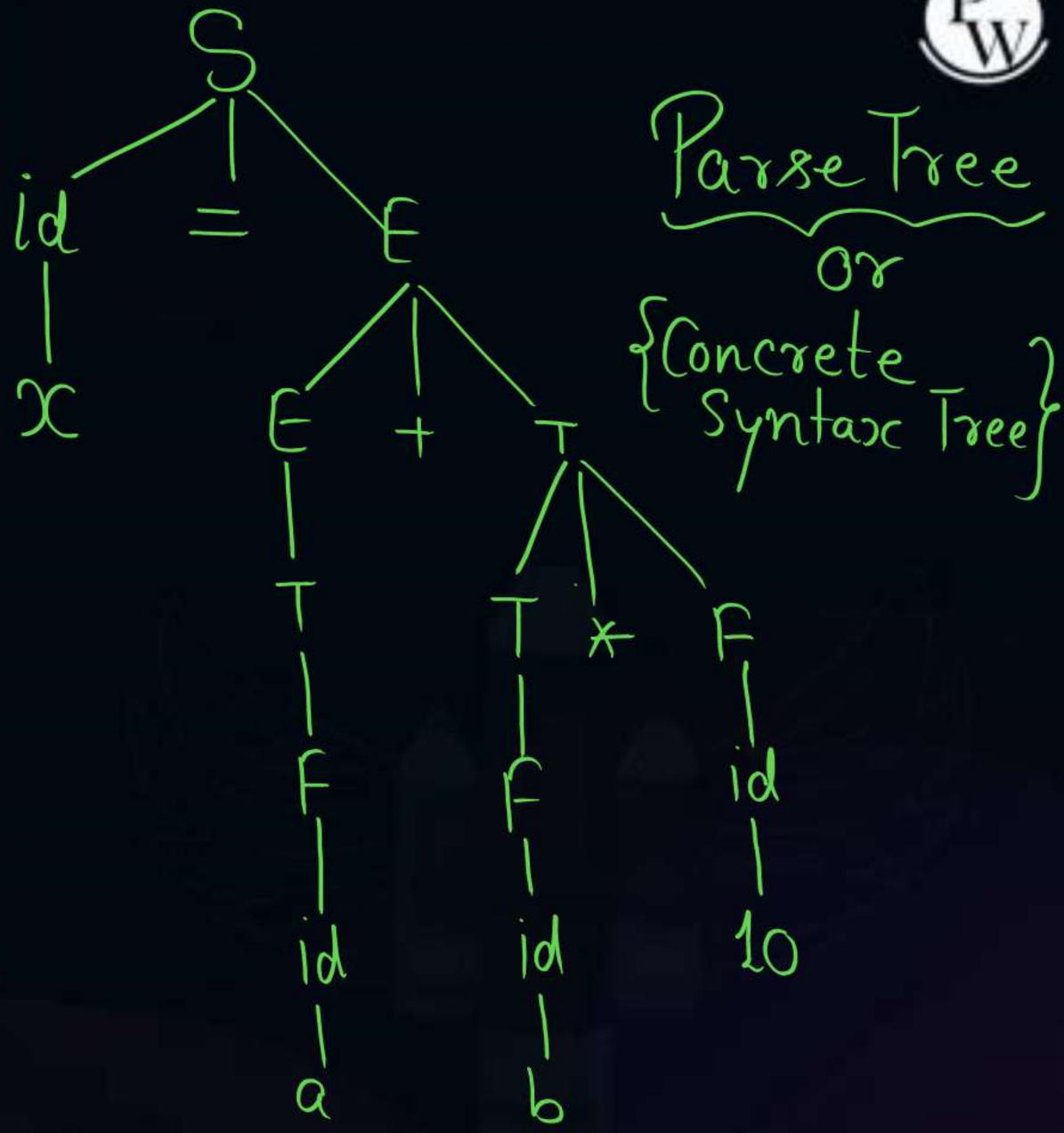
Syntax Analysis :

Already Known to Syntax Analyzer

- $S \rightarrow id = E$
- $E \rightarrow E + T \mid T$
- $T \rightarrow T * F \mid F$
- $F \rightarrow id$

String $x = a + b * 10$

id_1 id_2 id_3 Constant



Parse Tree or {Concrete Syntax Tree}

Slide

Semantic Analysis :

Symentic analyzer is used to check whether the statement is semantically valid or not {i.e. meaningful or not}

eg: Type checking is done by semantic analyzer

lets assume that above statement is sematically Correct.

Slide



Intermediate Code Generation :

$x = a + b * 10$

each expression contains atmost three add^r (operand)

Tree Add^r Code

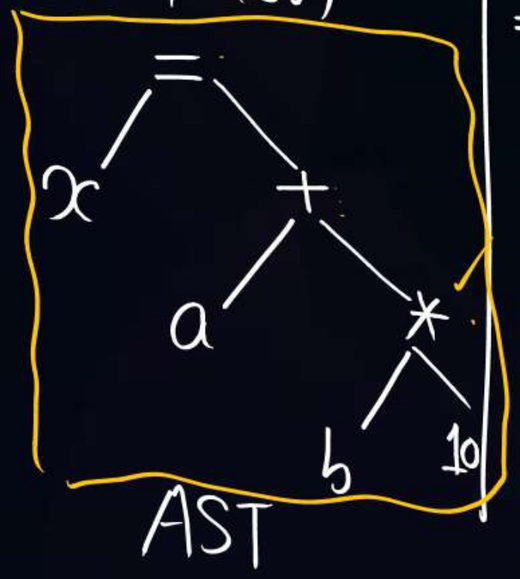
$t_1 = b * 10$

$t_2 = a + t_1$

$x = t_2$

Syntax Tree (or)

eg:



AST

Slide

$x = a + b * 10$

These are multiple intermediate representation

- Linear Form → Three add^r Code
- Tree form → Syntax Tree (or) Abstract Syntax Tree (AST)

{Code optimization is an optional Phase

In Code optimization we try to transform the Code such that it uses less time (or) less resources

Intermediate Code Optimization :

let, Three add^{r} Code is

(i) $t_1 = b * 10$

(ii) $t_2 = a + t_1$

(iii) $x = t_2$

Output of Intermediate Code generation phase

optimization

$t_1 = b * 10$

$x = a + t_1$

Output of Intermediate Code Optimization Phase

time required for the execution of (iii)rd statement is Saved

Slide

Let, $t_1 = b * 10$
$x = a + t_2$
} is given to Target code generation phase

{To be understood after COA}

Target Code Generation :

It is Machine dependent Phase {i.e. Architecture dependent}

$t_1 = b * 10$
$x = a + t_1$

Architecture is Three add^r Architecture

MUL R_1, b, #10
ADD x, a, R_1

Architecture is Register-Register Architecture

↓

leave for COA

Architecture is 2-Add^r Architecture

Mov R_1, b	$R_1 \leftarrow b$
MUL R_1, #10	$R_1 \leftarrow R_1 * 10$
Mov R_2, a	$R_2 \leftarrow a$
ADD R_2, R_1	$R_2 \leftarrow R_2 + R_1$
Mov X, R_2	$X \leftarrow R_2$

Slide

* Target Code optimization :-

↳ Not to be discussed w.r.t. GATE



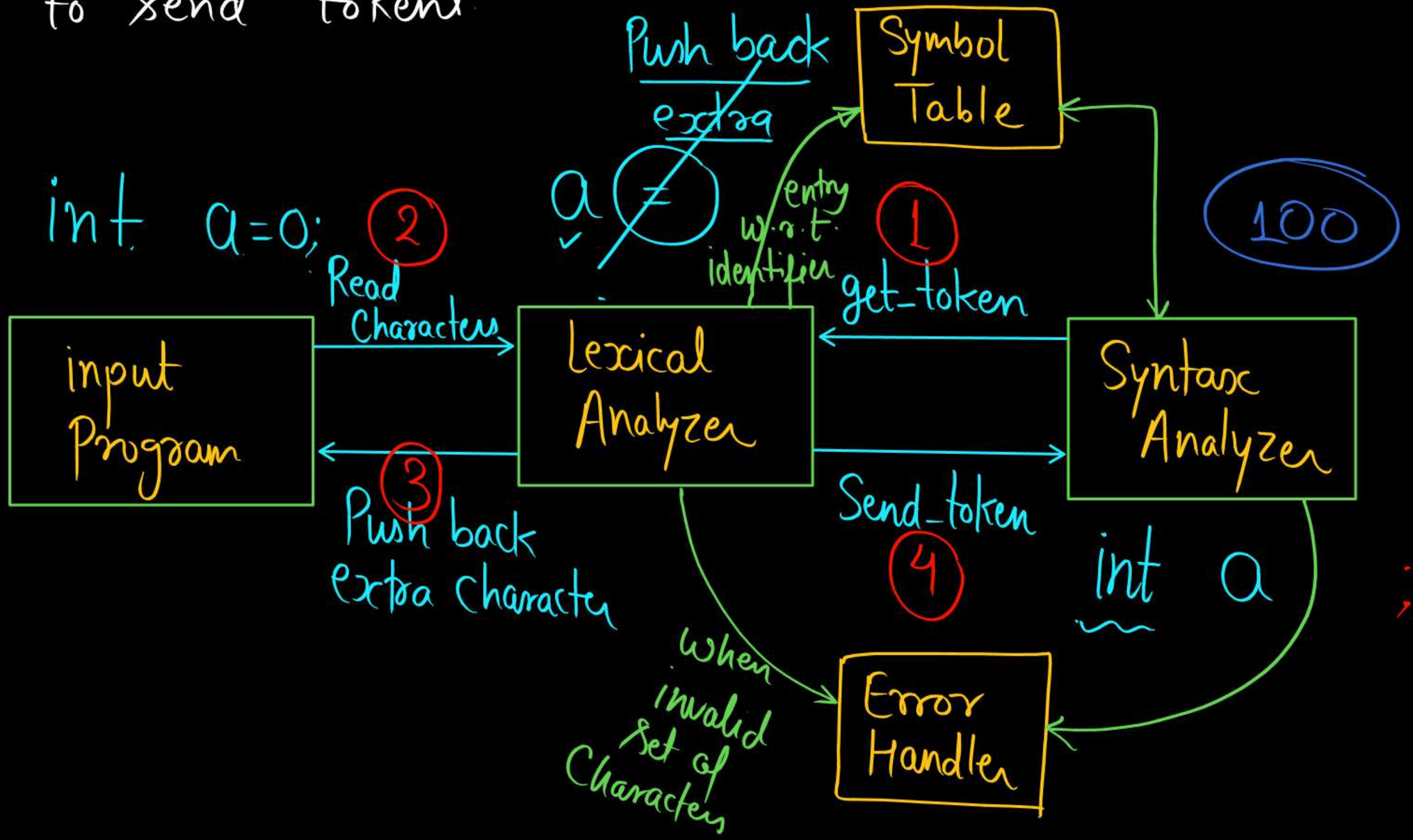
Topic : Lexical Analyzer

{ Convert stream of characters into stream of tokens }



Slide

* Syntax analyzer will request lexical analyzer to send tokens



Functionality of Lexical Analyzer :

1. Lexical Analyzer will scan the input program Character by Character and groups them into meaningful words called as lexemes and Categorizes them into tokens.
   * for tokenization it uses longest Prefix rule.
2. It generates lexical errors if any
3. It eliminates white spaces (blanks) and Comments.
4. It starts the Construction of symbol table } Partially, other phases will also update and use symbol table }
5. It also maintains line numbers

Slide

Types of tokens in C :

Home work

1. Keyword
2. Identifier
3. Operators
4. Special Symbols
5. Constant
6. String

Slide

1

Keywords : { Home work }

{ 32 - keywords }

Slide

(2) **Identifiers :** Home work

{ format and Rules }

Slide

Home work

3. * Operators
4. * Special Symbol
5. * Constant
6. * String

Slide



2 mins Summary

Topic Parse tree(Concrete Syntax tree) vs Syntax tree(Abstract Syntax Tree)

Topic Process of translation

Topic Basic understanding of lexical analyzer

Slide

THANK - YOU